
Training a Policy Gradient Agent for Coup via REINFORCE with Reward Shaping

Jacqueline Cai
(jxc6)

Yunshen(Harry) Song
(ys2388)

1 Introduction

This project creates an agent that utilizes Reinforcement Learning in the social deduction game, Coup. With the use of AI, the new agent outperforms previously used statistical agents. Coup is a multi-player card game based around bluffing and deduction in which players take turns claiming to have certain characters. Players can either be challenged on their claim by a fellow player or simply block the claim. The game is complete when one player retains all their influence cards while the others have lost both of theirs.

Due to the fact that Coup is purely episodic, has finite game outcomes consisting of either a win or loss, and has a finite number of possible actions that can be performed in a turn, it is a natural fit for policy gradient methods. We selected the REINFORCE algorithm Zhang et al. [2020] (a Monte Carlo-based policy gradient algorithm) because it optimizes the agent’s policy by storing the full game trajectories and updating the probabilities of taking different types of actions based on the discounted reward from the final game outcome. The REINFORCE algorithm is relatively simple, allowing for rapid iterations to be applied to the method while still having enough expressiveness to perform well in a game of Coup, which has a great deal of complexity in terms of strategic depth.

The agent is implemented as a multi-head neural network with 4 output heads, one for each decision type in Coup: choosing an action, deciding whether to challenge, deciding whether to block, and choosing a character to claim when blocking. We used a common set of hidden layers in the network that feed into each of the 4 heads, allowing the network to learn shared representations across all decision types while still modeling each decision independently. Training occurs at the end of each game using the full trajectory of decisions made, with weights saved to local storage so the agent continues learning across sessions. The following sections describe the state space encoding, reward function design, network architecture, and experimental results in detail. Our GitHub link is provided here.

2 Network Architecture

We built the neural network as a multi-head policy network. The input is the 73-dimensional state vector described in the previous section, which we feed through two fully connected hidden layers before splitting into four output heads, one per decision type:

- **Input layer:** 73 features
- **Hidden layer 1:** 64 units, ReLU activation
- **Hidden layer 2:** 32 units, ReLU activation
- **Action selection head:** 16 outputs (which action and target to choose)
- **Challenge head:** 2 outputs (allow or challenge)
- **Block head:** 2 outputs (allow or block)
- **Block claim head:** 5 outputs (which character to claim)

All heads produce raw logits, with softmax applied at sampling time so that invalid actions can be masked before the probability distribution is computed.

We trained the network using the Adam optimizer with a learning rate of 0.001. To prevent the policy from collapsing to deterministic behavior early in training, we added an entropy regularization term to the loss with a coefficient of 0.01. Rather than running a gradient update after every individual game, we batched training across games and fired one update every 10 games over the accumulated trajectory buffer. We also capped each game’s trajectory to the most recent 60 decisions to keep training stable and memory bounded.

3 State Space

The state of the game is represented in a single vector of fixed length of 73 dimensions that will be computed at each time the agent makes a decision (which means there will be multiple instances of this vector). This allows the agent to know everything a player could see in the game based on the player’s own perspective (4 distinct categories).

There are 40 of the 73 dimensions in the vector that encode information about the four player positions (all five players represented). Each of the four player positions has an 8 dimensional encoding (4 values each) for that player’s coin count (normalized to 12), their number of active influence cards (normalized to 2), if that player has been eliminated, and the count of each of the five characters that have been exposed to the public. Because these things are all available to any player at the table, the information in these dimensions gives the agent the complete picture of the public space.

There are 10 of the 73 dimensions in the vector that encode the agent’s own hand. For each of the two card positions, the agent has a 5 dimensional encoding (one-hot) of the five characters. Any exposed or missing character is represented as a vector of all zeros (5-dimensional vector of zeros). This hand is the only piece of true “private” information the agent has, which is very important when the agent is determining whether to bluff or how to block.

The dimensions from 50 to 68 show the current context of the decision being made. The index 68 is a single bit that indicates if it’s the agent’s turn or not. The index from 50-56 show the pending action in a 7 dimensional one-hot encoding, while 57-61 encode which player is the actor in a 5 dimensional one-hot. Additionally, 62-66 indicate the target using a 5 dimensional one-hot and a bit for presence. As a whole, this provides full context about what is happening to the agent at any given moment, which is most important to the decisions made regarding challenge, block, and claim a block.

The dimensions from 69-72 consist of a 4 dimensional one-hot encoding defining the type of decision being made, with an index for: selecting an action, challenging, blocking, and selecting a block claim. These represent four different types of decisions which are obtained by feeding the same encoder into all four heads of decision output to communicate to the network what the decision type is and allow the network to learn different behaviour depending on the decision context using the same underlying representation.

4 Action Head and Validity Masking

The 16-dimensional action selection head uses a fixed action-target encoding. The first four output slots correspond to non-targeted actions: income, foreign aid, tax, and exchange. The remaining twelve slots represent the three targeted actions, coup, assassinate, and steal, paired with up to four possible opponents. This gives the policy a consistent output space even though the number of legal targets changes as players are eliminated.

Because not every output is legal in every state, we construct a validity mask before sampling from the action head. For example, coup is only valid when the player has at least 7 coins, assassinate is only valid with at least 3 coins, and when a player has 10 or more coins, the mask forces the policy to choose coup. Target slots corresponding to eliminated or nonexistent opponents are also masked out. Invalid logits are assigned a large negative value before the softmax is computed, which gives them effectively zero probability.

The same masking idea is used for the other heads. The challenge and block heads each have two outputs, corresponding to allow versus challenge or allow versus block. The block-claim head has five outputs, one for each character, but its mask only allows characters that can legally block the current action. For example, foreign aid can only be blocked by Duke, assassinate by Contessa, and

steal by either Captain or Ambassador. We also store the valid mask with each sampled decision so that the training loss is computed over the same legal action space that was available during gameplay.

5 Reward Function

Our reward function gives us one terminal reward per episode and it is defined as follows:

$$R = \text{base} + (0.02 \times \text{coins}) + (0.01 \times \text{turns survived})$$

Here `base` is equal to `+1.0` if the agent wins and `-1.0` if the agent loses, `coins` is the number of coins owned by the agent at the end of the game, and `turns survived` is the number of turns taken by the agent during the game. These signals were chosen based on our observations because they seemed the most meaningful indicators of good playing that we could receive in the terminal state. The first term encourages gathering resources that become very helpful in terms of gaining access to stronger actions in Coup, including coup and assassination. The second term rewards staying in the game longer, which provides a useful gradient signal in the majority of the games the agent loses early in training.

Another approach we tried was adding intermediate rewards based on the occurrence of opponent card losses, where a small positive reinforcement was added whenever the agent helped in removing influence from an opponent. From a common-sense perspective, this seemed to be a logical reinforcement to add, because card removal was the very essence of how our agent could win in the game. But in reality, this proved to be overwhelming compared to the terminal reward, resulting in the agent playing too aggressively in terms of card-loss events and not in other aspects of the game, leading to poorer performance compared to the simpler terminal reward model.

6 Evaluation

After training, the neural network agents were evaluated in three-player Coup simulations against one Statistical AI agent and one Challenger AI agent. Each evaluation was run for 10,000 simulated games. Two neural network architectures were tested: a smaller 32/16 hidden-layer model and a larger 64/32 hidden-layer model.

The smaller 32/16 neural network achieved the strongest result in its evaluation, winning 5,479 out of 10,000 games, corresponding to a win rate of 54.8%. The Statistical AI won 2,525 games, or 25.3%, while the Challenger AI won 1,996 games, or 20.0%.

Agent	Wins	Win Rate	Avg. Coins	Avg. Turns Survived
Neural AI (32/16)	5479	54.8%	2.0	27.5
Statistical AI	2525	25.3%	0.3	25.6
Challenger AI	1996	20.0%	0.3	16.2

Table 1: Evaluation results for the Neural AI 32/16 model over 10,000 games against Statistical AI and Challenger AI.

The larger 64/32 neural network performed even better in the later evaluation recorded in the game logs. Over 10,000 games, it won 6,266 games, giving it a win rate of 62.7%. The Statistical AI won 2,313 games, or 23.1%, while the Challenger AI won 1,421 games, or 14.2%.

Agent	Wins	Win Rate	Avg. Coins	Avg. Turns Survived
Neural AI (64/32)	6266	62.7%	0.8	21.0
Statistical AI	2313	23.1%	0.3	19.8
Challenger AI	1421	14.2%	0.2	13.9

Table 2: Evaluation results for the Neural AI 64/32 model over 10,000 games against Statistical AI and Challenger AI.

These results show that both neural network agents outperformed the baseline agents. Since there were three agents in each game, an even distribution would give each agent an expected win rate of about 33.3%. Both neural agents exceeded this baseline, with the 32/16 model reaching 54.8%

and the 64/32 model reaching 62.7%. In this evaluation, the larger 64/32 architecture produced the stronger policy.

The game logs suggest that the 64/32 Neural AI learned a low-risk, exploitative strategy. Its most common actions were foreign aid, income, coup, and tax. It used foreign aid 48,654 times, income 16,829 times, coup 12,120 times, and tax 8,742 times. In contrast, it rarely used the more interactive actions: assassinate 89 times, exchange 35 times, and steal 30 times. This indicates that the agent generally avoided risky actions that could be challenged or blocked, preferring to build coins safely and finish opponents with coup.

A key pattern in the logs is that the Neural AI almost never challenged. It initiated only 15 challenges, all of which failed. By contrast, the Statistical AI failed 7,441 challenges and the Challenger AI failed 8,366 challenges. This suggests that a major part of the Neural AI's advantage came from avoiding unnecessary challenge risk while the other agents frequently lost influence through failed challenges.

The Neural AI also made effective use of defensive blocking. It blocked 34,480 opponent actions, including 20,534 ambassador blocks and 8,735 contessa blocks. Some of these blocks were false claims, but the opponents did not punish them consistently enough to remove the advantage. As a result, the Neural AI was often able to survive blockable attacks, accumulate coins, and use coup as a reliable finishing action.

Overall, the Neural AI appears to have learned an exploitative but effective policy for this matchup: avoid risky challenges, rely on safe economy actions, use mostly reliable tax opportunities, block aggressively on defense, and win through repeated coups. However, this also shows a limitation of the evaluation. The high win rate demonstrates strong performance against the Statistical and Challenger agents, but it does not necessarily prove that the learned policy would generalize equally well against stronger agents or human players that challenge defensive bluffs more selectively.

6.1 COUP Competition Results

We also held a competition between the student COUP agents developed this semester. Since the results varied significantly depending on the matchup and number of players, there was no single clear winner across all settings. When averaging performance across the recorded simulations, our Neural (64/32) agent achieved the highest average win rate at **45.5%**, slightly ahead of Raj_NickAI at **44.9%** and well above PPO at **17.7%**. Each matchup was evaluated using 1000-game Monte Carlo simulations, which revealed that agent performance depended heavily on the opponent pool and game setting.

Raj_NickAI was the strongest head-to-head competitor, defeating PPO with a 75.0% win rate and also beating Neural (64/32) with a 61.1% win rate. PPO struggled in direct matchups, especially against Neural (64/32), where it won only 0.5% of games, but it performed better in larger mixed-agent games, winning the 4-player and 5-player simulations with 31.4% and 29.3% win rates, respectively. Our Neural (64/32) agent was especially effective against PPO, winning 99.5% of their direct matchup, and it remained competitive across the broader simulations.

Overall, the competition showed that no single agent dominated every setting. Raj_NickAI performed best in direct matchups, PPO was more competitive in multi-agent environments, and our Neural (64/32) agent had the strongest average performance across the recorded results. These results suggest that the relative strength of each agent was highly matchup-dependent rather than determined by a single overall ranking.



Figure 1: COUP Contest Between All Agents

7 Learnings

One lesson learned was that the network architecture only works well if both the input and output representations match the structure of the game. Coup includes hidden information, public information, changing player counts, and several different decision types, so the 73-dimensional state vector needed to include enough context for each decision, such as the pending action and target player. Similarly, the 16-dimensional action head gave the policy a consistent output space, but it required validity masking so the agent could not choose illegal moves, such as assassinating without enough coins or targeting an eliminated player.

Our second major lesson was the realization that keeping a simple reward scheme will often perform better than a complicated one. A number of papers related to reinforcement learning have explored complex reward shaping and using many intermediate reward signals, so we initially went in this direction by creating opponent card loss events to include as a reward in our design but ended up destabilizing training, which resulted in having lower win rates than our simpler baseline. Instead, we found that having a clear terminal reward with only two small shaped bonuses was successful in our case, which leads us to believe there is a broader lesson in RL where using a reward defined more closely to the actual objective of the RL system will outperform one that is based on more rewards with unintended incentives.

References

Junzi Zhang, Jongho Kim, Brendan O’Donoghue, and Stephen Boyd. Sample efficient reinforcement learning with reinforce, 2020. URL <https://arxiv.org/abs/2010.11364>.